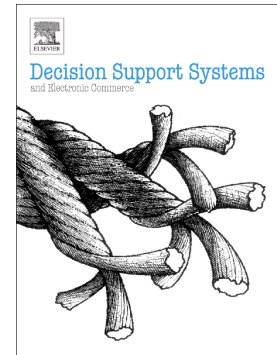


Automated discovery of business process simulation models from event logs

Manuel Camargo, Marlon Dumas, Oscar González-Rojas



PII: S0167-9236(20)30039-7

DOI: <https://doi.org/10.1016/j.dss.2020.113284>

Reference: DECSUP 113284

To appear in: *Decision Support Systems*

Received date: 18 October 2019

Revised date: 27 February 2020

Accepted date: 12 March 2020

Please cite this article as: M. Camargo, M. Dumas and O. González-Rojas, Automated discovery of business process simulation models from event logs, *Decision Support Systems* (2020), <https://doi.org/10.1016/j.dss.2020.113284>

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Automated Discovery of Business Process Simulation Models from Event Logs

Manuel Camargo^{a,b,*}, Marlon Dumas^a, Oscar González-Rojas^b

^a *University of Tartu, Tartu, Estonia*

^b *Universidad de los Andes, Bogotá, Colombia*

Abstract

Simulation is a versatile technique for quantitative analysis of business processes. It allows analysts to estimate the performance of a process under multiple scenarios. However, the discovery, validation, and tuning of business process simulation models is cumbersome and error-prone. It requires manual iterative refinement of the process model and simulation parameters in order to match the observed reality as closely as possible. Modern information systems store detailed execution logs of the business processes they support. Previous work has shown that these logs can be used to discover simulation models. However, existing methods for log-based discovery of simulation models do not seek to optimize the accuracy of the resulting models. Instead they leave it to the user to manually tune the simulation model to achieve the desired level of accuracy. This article presents an accuracy optimized method to discover business process simulation models from execution logs. The method decomposes the problem into a series of steps with associated configuration parameters. A hyper-parameter optimization method is then used to search through the space of possible configurations so as to maximize the similarity between the behavior of the simulation model and the behavior observed in the log. The method has been implemented as a tool and evaluated using logs from different domains.

Keywords: Process simulation, Process mining, Automated process discovery

* Corresponding author

Email addresses: manuel.camargo@ut.ee (Manuel Camargo), marlon.dumas@ut.ee (Marlon Dumas), o-gonzal@uniandes.edu.co (Oscar González-Rojas)

1. Introduction

Business Process Simulation (BPS) is a widely used technique for quantitative analysis of business processes [1]. The main idea of BPS is to generate a set of possible execution traces of a process from a business process model annotated with parameters such as the arrival rate of new process instances, the processing time of each activity, etc. The resulting execution traces are then used to compute performance measures of the process, for example, cycle time, resource utilization, and waiting times for each task in the process.

A key ingredient for BPS is the availability of a simulation model (herein the BPS model) that accurately reflects the actual dynamics of the process. Traditionally, BPS models are created by domain experts using manual data gathering techniques, such as interviews, contextual inquiries, and on-site observation. This approach is time-consuming [2]. Furthermore, the accuracy of a BPS model discovered in this way is limited by the accuracy of the business process model that is used as a starting point. Yet, oftentimes process models produced by domain experts do not capture all possible execution paths (e.g. exceptional paths are left aside). Indeed, given that business process models are often designed for documentation and communication purposes, they need to strike a balance between completeness and understandability.

Previous studies have advocated the use of Process Mining (PM) techniques to discover BPS models from business process execution logs (also known as *event logs*) [3, 4]. The key idea behind these studies is that a business process simulation model can be obtained by first extracting a process model from an event log using an *automated process discovery* technique, and then enhancing this model with simulation parameters derived from the event log (e.g. arrival rate, processing times, and conditional branching probabilities).

However, existing proposals in this area do not consider the question of measuring and automatically tuning the accuracy of the resulting BPS models. Instead, existing proposals rely on user intervention to manually tune the resulting BPS models throughout the different steps in their construction.

This article addresses this gap by proposing an accuracy optimized method to automatically discover BPS models from event logs. The method decomposes the problem at hand into a series of steps, each of which can be configured via one or more hyper-parameters.¹ A

¹ The term *hyper-parameter* is used to refer to any of the parameters of the method for discovering BPS models,

hyper-parameter optimization method is used to search through the space of possible configurations of the BPS discovery method so as to maximize the similarity between the behavior of the BPS model and the behavior observed in the log. In order to guide this optimization procedure, this article puts forward a measure of accuracy for log-derived BPS models.

The proposed method has been implemented as a tool (namely Simod) that generates process simulation models from an event log in the eXtensible Event Stream (XES) format. The resulting simulation model that can be executed using two simulators: BIMP [5] and Scylla [6]. The magnitude of the accuracy enhancements achieved by the proposed method have been evaluated via experiments on three event logs from different domains.

This article is a significantly extended version of a tool demonstration paper [7]. The previous tool paper outlines the high-level architecture of Simod. This article adds a detailed description of the algorithms employed, a definition of the BPS model accuracy measure, and an evaluation of the proposed method.

The rest of the article is structured as follows. Section 2 introduces basic terminology in the field of BPS and discusses existing approaches for BPS model discovery. Section 3 presents the proposed method and the approach for measuring BPS model accuracy. Section 4 discusses the experimental evaluation. Finally, Section 6 draws conclusions and outlines directions for future work.

2. Background and Related Work

This section presents background concepts used throughout the article, followed by an analysis of the related work in data-driven BPS.

2.1. Business process simulation

This article considers business process models represented in the Business Process Model and Notation (BPMN). In its basic form, a BPMN process model consists of a set of activity nodes (or *activities* for short) and gateways that are interconnected by sequence flows. A *split gateway* has multiple outgoing sequence flows. An *exclusive decision gateway* is a split gateway that

while the term *parameter* refers to any of the parameters of the generated BPS model, such as the arrival rate or the processing times of activities.

encodes one decision, i.e. when the execution of the process reaches this gateway only one of its outgoing sequences flows is taken. An *inclusive decision gateway* allows multiple of its outgoing flows to be taken when encoded decisions are satisfied. However, any inclusive decision gateway can be trivially transformed into a combination of exclusive decision gateways and parallel gateways, hence we can restrict ourselves to exclusive decision gateways without loss of generality. The branches coming out of a decision gateway are called *conditional branches*.

The *cycle time* of a process instance (herein called a *case*) is the amount of time between the moment the case starts its execution and the moment it ends. By extension, we can also define the cycle time of an instance of an activity as the amount of time between the moment the activity instance is enabled (i.e. ready to be executed) and the moment it completes. The *processing time* of an activity instance is the amount of time between the moment the activity instance is started and the moment it is completed. Usually, there is a delay between the moment an activity instance is enabled and the moment it starts. This delay is called *waiting time*. Again by extension, we define the processing time of a case as the amount of time when the process instance is *active*, meaning that at least one activity instance of this case has started but not yet completed. The *waiting time* of a case is the cycle time of the case minus the processing time. These definitions also apply to a process, which consists of a set of observed cases. The cycle time of a process is the mean cycle time of its cases. Similarly, the cycle time of an activity is the mean cycle time of its activity instances.

Given the above terminology, a BPS model consists of a process model plus the following elements [1]:

- The *mean inter-arrival time* of cases and its associated probability distribution function, e.g. one case is created every 10 seconds on average with an exponential distribution.
- The probability distribution of the processing times of each activity. For example, the processing times of an activity may follow a normal distribution with a mean of 20 minutes and a standard deviation of 5 minutes, or an exponential distribution with a mean of 10 minutes.
- For each conditional branch in the process model, a *branching probability* (i.e. percentage of time the conditional branch in question is taken when the corresponding decision gateway is reached).
- The *resource pool* that is responsible for performing each activity in the process model. For

example, in an insurance claims handling process, a possible resource pool would be the *claim handlers*. Each resource pool has a size (e.g., the number of claim handlers or the number of clerks). The instances of a resource pool are the *resources*.

- A *timetable* for each resource pool, indicating the time periods during which a resource of a resource pool is available to perform activities in the process (e.g. Monday-Friday from 9:00 to 17:00).
- A function that maps each task in the process model to a resource pool.

A BPS model consisting of the above elements can be executed using a discrete event simulator. The simulation of a BPS model yields a simulation log, consisting of the execution traces generated during the execution, as well as a collection of performance measures (e.g. cycle time, resource utilization, etc.).

2.2. *Related work*

Data-driven approaches to BPS can be classified in two categories. The first category consists of approaches that provide conceptual guidance to discover BPS models. The second category consists of approaches that seek to automate the discovery of BPS models. Below we review each of these two categories.

Conceptual guidance for data-driven simulation. These approaches discuss how process mining techniques can be used to extract, validate and tune BPS model parameters, without seeking to provide fully automated support.

The authors in [8] identify four main components of a simulation model, namely entities, activities, resources and gateways. The authors then identify BPS modeling tasks related to each of these components (e.g., modeling gateway routing logic, modeling activities). Later, in [4] the same authors present a literature review on the use of process mining techniques to support each of these modeling tasks. This literature review sheds insights into the question of how to choose process mining techniques for each of the BPS modeling tasks. In this paper, we use these insights as a starting point for the design of an automated method for discovery of BPS models from event logs.

In [9], the authors present an approach to enhance a given process model with simulation parameters. This approach differs from the one presented in the present article in that it assumes

that process model is given as input (in addition to the event log). The approach also assumes that the process model perfectly fits the event log. In reality though, the traces in the event log may deviate with respect to the behavior captured in the process model. Moreover, the approach in [9] does not seek to provide an automated end-to-end approach for discovering BPS models from event logs, but it rather focuses on providing guidance for approaching some of the steps in the discovery of a BPS model.

The authors in [2] present a methodology for process improvement based on data-driven simulation. The authors propose as a first step the discovery of simulation models from data for the representation of the current state of the process. Next, the authors propose the manual evaluation of possible scenarios to lead the process to a desired state. The work is illustrated with three case studies from the gas, government and agriculture industries. This work is useful for scoping the relevance of using data for discovering simulation models, as well as for identifying the need for automating this task to explore possible scenarios more quickly and efficiently. However, it does not provide concrete guidance for discovering accurate BPS models from process execution data.

Automated discovery of BPS models using process mining. The methods in this category seek to automate the discovery of BPS models from event logs by means of process mining techniques.

Rozinat et al. [10] propose a semi-automatic approach to discover BPS models based on Colored Petri Nets (CPNs). In this work, an event log is used as input for the discovery of various elements of a BPS model, including the control-flow structure (i.e. the process model), the conditional branching probabilities, and the resource pools. However, the automatic discovery of activity processing times and case inter-arrival times (and their probability distributions) are left aside. In [3], the authors go further by proposing a technique to discover more complete BPS models that include processing times and case inter-arrival times. These simulation parameters are then combined with the process model into a single CPN, which can be simulated using a CPN tool.

One limitation of the work of Rozinat et al. [3] is that it does not seek to automatically adjust or fit the probability distributions of the processing times of activities (nor the probability distribution of case inter-arrival times). Also, although most of the steps in the BPS model discovery are automated, the step where the multiple model BPS model elements are merged together is not automated. Moreover, Rozinat et al. do not seek to optimize the accuracy of the

resulting BPS model, nor does it propose a comprehensive approach to measure the accuracy of a simulation model with respect to an event log. The authors suggest to measure the accuracy of the simulation model by comparing the cycle time produced by the BPS model to the ground truth, but this only provides a coarse-grained assessment. Two event logs may have similar cycle times, yet the activities in the corresponding traces may occur at very different points in time and possibly also in different order.

Khodyrev et al. [11] propose a process mining approach to generate BPS models tailored for short-term prediction of performance measures. The authors extract the structure of the process as a Petri net and the dependencies between elements and variables are established using decision trees. Subsequently, two experiments were conducted with real events logs, in which the authors sought to predict specific performance measures for each process, such as the number of open or closed events during the prediction period, the processing or cycle times. A major limitation of this approach is that it does not discover the resource perspective (i.e. the resource pools) of the BPS model. Instead, it is assumed that each activity may be performed by an infinite amount of resources. Also, while the discovery of the conditional branching probabilities and of the activity processing times are done automatically, the integration of these elements into a BPS model is left to the user. This latter approach does not define how to measure and optimize the accuracy of the resulting BPS model.

Finally, Gawin et al. [12] combine multiple process mining techniques to create a BPS model that reflects the actual process behavior. Specifically, process mining techniques are employed to extract the process model structure, the resource pools, the activity processing times, and the decision logic of decision gates. Interviews and process documentation techniques are used to elicit the case inter-arrival times, the costs of resources use, and the definition of resource schedules. The simulation parameters discovered in this way are then manually linked in the ADONIS tool, leading to a BPS model that is then executed with a capacity analysis algorithm of this latter tool. This latter work differs from the one reported in this article in that it does not seek to automate the extraction of all elements of a BPS simulation model (nor their assembly). Also, it does not seek to measure and optimize the accuracy of the BPS model.

Table 1 summarizes the capabilities of the above approaches for BPS model discovery. In this table, the symbol (+) implies that the feature is supported, (–) implies not supported, and (+ / –) implies partially supported, for example, supported but not in an automated manner.

Characteristics	Rozinat et al. (2006)[10]	Rozinat et al. (2009)[3]	Khodyrev et al. (2014)[11]	Gawin et al. (2015)[12]
Sequence flow discovery	(+)	(+)	(+)	(+)
Resource pools discovery	(+)	(+)	(-)	(+)
Branching probabilities discovery	(+)	(+)	(+ / -)	(+)
Probabilities distribution fitting	(-)	(-)	(-)	(-)
Model assembly	(-)	(-)	(-)	(-)
Accuracy assessment	(-)	(+ / -)	(+ / -)	(+ / -)
Accuracy optimization	(-)	(-)	(-)	(-)

Table 1: Comparison of approaches to discover and/or enhance BPS models

This article advances the state-of-the-art in two ways. First, it proposes a fully automated method for discovering each of the perspectives of a BPS model and assembling the resulting perspectives into a complete BPS model. Second, it proposes an approach to measure the accuracy of a BPS model and to optimize the accuracy of an automatically discovered BPS model.

3. Approach

The proposed method takes as input an event log in which every event (corresponding to the execution of an activity instance) has a case identifier, an activity label, a resource attribute (indicating which resource performed the activity), and two timestamps: the start timestamp and the end timestamp². The resource attribute is required to discover the available resource pools, their timetables, and the mapping between activities and resource pools. Equally the start and end timestamps are required to compute the processing time of activities and their respective probability distributions. Figure 1 illustrates the steps of the proposed method and their inputs and outputs.

² Alternatively, we can also take as input an event log where each activity instance is recorded via two events: a start event and an end event, each one with a corresponding timestamp. The event log file format can be either XES format or simply in CSV format

Figure 1: Steps of the BPS model discovery method

The following describes these capabilities and exemplifies them by using a synthetic event log of a purchase-to-pay (P2P) process. This event log ³ meets the aforementioned requirements and it consists of 21 activities, 27 resources, and 9119 events related to 608 cases.

3.1. Pre-processing

In this step, an automated process discovery technique is applied to extract a BPMN process model from the event log. Most of the times and due to the characteristics of the process discovery algorithms, the discovered models do not reflect 100% of the possible paths that can be taken in a business process (in other words, the fitness is not 100%). Therefore, one of the main concerns at this stage is avoid the distortion of the observed reality. To this end, the proposed method also measures the conformance between the discovered BPMN model and the event log, and provides the possibility of applying repair actions on the log in order to improve the fitness between the model and the log.

Control Flow Discovery. This step is critical since it defines the activities, the decision gateways, and the way these are related in the process. We use the Split Miner algorithm [13] to generate BPMN v2.0 models from event logs. We selected this automated process discovery method because it achieves high levels of accuracy (precision and fitness) while at the same time producing simple process models, relative to other state-of-the-art process discovery techniques [14]. However, other process discovery methods such as the inductive miner, could be used instead [15]. Split Miner allows the discovery of models with different levels of sensitivity, which depends on the parameters epsilon (ϵ) and eta (η). The parameter ϵ refers to the parallelism threshold and determines the quantity of concurrent relations between events to be captured. ϵ is defined in a range between 0 and 1, and the larger the value of this parameter, the greater the number of possible relationships between the different events to be considered in the analysis. On the other hand, η refers to the percentile for frequency threshold, acting as a filter over the incoming and outgoing edges of each node retaining only the η percentiles most frequent. η is defined in a

³ Taken from the tutorial of the Disco process mining tool, available at: <http://fluxicon.com/academic/material/>

range between 0 and 1 and the larger the value of this parameter, the greater the percentage of frequencies to be retained. For the tool implementation, we used the command-line version of the Split Miner, which takes as input an event log in XES format and outputs a BPMN process model. Table 2 outlines the structure of this log, while Figure 2 illustrates the resulting model using ϵ as 0.3 and η as 0.7.

Case ID	Activity	Start Timestamp	Complete Timestamp	Resource
1	Create Purchase Requisition	2011/01/01 00:00:00	2011/01/01 00:37:00	Kim Passa
2	Create Purchase Requisition	2011/01/01 00:16:00	2011/01/01 00:29:00	Immanuel Karagianni
3	Create Purchase Requisition	2011/01/01 02:23:00	2011/01/01 03:03:00	Kim Passa
1	Create Request for Quotation	2011/01/01 05:37:00	2011/01/01 05:45:00	Kim Passa
1	Analyze Request for Quotation	2011/01/01 06:41:00	2011/01/01 06:55:00	Karel de G root
2	Create Request for Quotation	2011/01/01 08:16:00	2011/01/01 08:26:00	Alberto Duport
4	Create Purchase Requisition	2011/01/01 08:39:00	2011/01/01 09:00:00	Fjodor Kowalski

Table 2: Event-log format example

Figure 2: SplitMiner BPMN output example of the purchasing process

Alignment Evaluation. We measure the degree to which each trace in the log can be aligned with a corresponding trace produced by the process model by using the *fitness measure* proposed in [16]. This alignment is a sequence that has the length of the longest trace and it consists of three symbols: SM (“synchronous move”), MM (“move-on-model”) and ML (“move-on-log”). A SM indicates that the two traces match (i.e. the current activity is the same in both traces). A MM means that the two current activities do not match and that the algorithm will “skip” the current activity in the model. Thus, the algorithm moves forward in the trace of the model but we stay in the current position in the trace of the log. Conversely, a ML means that the two current activities do not match, thus the current activity is *skipped* in the trace of the log to align the two traces (and remain in the same position in the trace of the model). A perfectly aligned pair of traces contains only SM symbols. Otherwise, the number of MM and ML symbols capture the level of misalignment.

Log Repair. Once conformance (fitness) is measured, the model repair, event log repair or both can be performed to improve conformance between the event log and the model as is explained in [17]. In our case we perform the log repair, for which we propose the removal, replacement, or the repair of those traces in the log that do not fully fit a trace in the model.

Removal. This method omits the traces that are not in conformance with the extracted model, leaving only a reduced log composed of conformant traces. This method is the most natural, and computationally cheap to avoid the outlier traces in the data source. However, if the event log has low conformance with the model, it could end up with a very small event log that could not be sufficiently representative of the process dynamics being only valid for some cases.

Replacement. This method replaces each non-conformant trace with a copy of the most similar conformant trace. This action keeps the amount of traces of the log, and globally compensate the lack of alignment between the log of events and the model. The similarity between the traces is defined as one minus the normalized Damerau-Levenshtein (DL) distance between two strings. Therefore, we create an alphabet by assigning a unique character to each activity in the event log. This alphabet allows us to construct words that describe the order of execution of activities. Then each non-conformant trace is compared with all the conformant ones to find the closest one. Finally, the non-conformant trace is replaced in the event log by a copy of the most similar one.

Repair. This method repairs the event log in a local way by aligning non-conformant traces. This action keeps the maximum recorded observations preserving the recorded time variability. We use the automata-based alignment technique proposed in the ProConformance 2.0 tool⁴ to determine the optimal alignment of each process trace with the extracted model. This technique implements the conceptual ideas proposed in [16]. The automata-based representation of the event log speeds up the computation of the trace alignments in some cases. The tool implementation takes as input a process model and an event log and produces a set of alignments: one alignment for each trace in the log that does not perfectly align with the corresponding trace in the model. The alignments produced by this technique are optimal in the sense that each of them contains a minimum number of MMs plus MLs.

⁴ <http://apromore.org/wp-content/uploads/2017/04/ProConformance2.zip>

To repair a given trace, its corresponding trace alignment is scanned from left to right to apply one of two operations when an MM or an ML is found. The event in the trace responsible for an ML is removed. Conversely, the log is annotated with zero processing time and a special resource called AUTO when a MM is found. This means that this activity does not consume any resources and hence does not have an impact on the cycle time of the process. Finally, the algorithm advances one step in the trace alignment. When a SM is found.

Figure 3 presents a non-conformant trace and its corresponding repaired trace to illustrate the output of this trace alignment technique. For example, the original trace corresponds to the trace with case ID 100 from the purchasing process event log described in the running example. This trace has three activities, ending prematurely with respect to the process model discovered from the log. This trace corresponds to an occasional behavior where the business process is aborted after the first few activities. In this case, the ProConformance tool returns a fitness value of 0.4/1 and suggests a type of alignment that can be used to repair the event log: a synchronous move (SM), a move-on-model (MM), or a move-on log (ML).

Figure 3: Example: repairing a non-conformant trace

3.2. Processing

At this stage, the tool extracts the simulation parameters and assembles them with the process structure to create a BPS model. Below we discuss how each simulation parameter is computed.

Replay Event Log. Once a BPMN process model is obtained, the next step is to replay each trace in the repaired event log against the process model. This is done to calculate the *processing time* of each activity execution recorded in the trace and the *traversal frequency* of each conditional branch (i.e. the number of times that the conditional branch is traversed while replaying the trace in the log). These measures are later used to calculate the simulation parameters.

While replaying the repaired log against the model, the tool calculates the *enablement time* of each activity execution (i.e. event) recorded in the log. The enablement time of an activity execution is the moment in time when this activity was ready to be executed (i.e. the time when the state of the execution allowed this activity to start). In the simplest case, the enablement time of an

activity execution is equal to the end time of the activity that immediately precedes it in the trace, but this is not always the case in the presence of parallel activities in the process model. Then, the tool calculates the waiting time of each activity by subtracting the start time of activity A minus the enablement time of A. The waiting times calculated in this way are later compared to the waiting times calculated by the simulator in order to determine the accuracy of the simulation. The replay procedure is described in Algorithm 1.

ALGORITHM 1 Replay

inputs : A Process Model M, A trace T
output : processingTime: A map from events in T to Int
output : enablementTime: A map from events in T to Timestamp
output : traversalFrequency: A map from sequence flows in M to Int

for each ($e \in T$) {
 $processingTime[e] \leftarrow endTime(e) - startTime(e);$
 repeat
 $gatewayFired \leftarrow false;$
 for each ($g \in gateways(M)$) {
 if $isEnabled(M, currentMarking, g)$ **then**
 if $gatewayType(G) = XOR$ **then**
 $tcf \leftarrow traversedConditionalFlow(M, currentMarking, g, e);$
 $traversalFrequency[tcf] ++;$
 $currentMarking \leftarrow fire(M, currentMarking, g, e);$
 $gatewayFired \leftarrow true;$
 end
 end
 }
 until not $gatewayFired;$
 for each ($t \in Tasks(M)$ **where** $isEnabled(M, currentMarking, g)$) {
 if ($enablementTime[nextOccurrence(t, T)] \neq \emptyset$) **then**
 $enablementTime[nextOccurrence(t, T)] \leftarrow currentTime;$
 end

```

}
currentMarking  $\leftarrow$  fire( $M$ , currentMarking,  $e$ );
currentTime  $\leftarrow$  endTime( $e$ );
}
return processingTime, enablementTime, traversalFrequency;

```

The replay algorithm takes as input a process model and a trace, and calculates the processing time and the enablement time of each activity execution (event) in the trace, as well as the traversal frequency of each conditional flow traversed while replaying the trace. Once the traversal frequencies for each trace are computed, the tool sums up the traversal frequencies across all traces in the log in order to compute the total traversal frequency of each conditional flow. The algorithm relies on the concept of *marking of a BPMN process model* [18] to capture an execution state with respect to a BPMN model. In a semantically correct (sound) BPMN process model, a marking is a function that maps each sequence flow in the model to a boolean. A sequence flow is mapped to true if and only if, in the current state, there is a token in that sequence flow.

The algorithm initializes the current marking as the initial marking of the model (i.e. the marking where there is a token in the sequence flow coming out of the start event of the model). Then, the algorithm iterates over each event in the input trace. Each event denotes an execution of an activity, and has a start time and an end time, which allows us to calculate the processing time of the activity. Before handling a given event e , the algorithm fires every gateway that is enabled in the current marking, until no more gateways can be fired. When an XOR-gateway is fired, the conditional flow that leads to an activity corresponding to event e is traversed. Accordingly, the traversal frequency of this conditional flow is increased by one.

Next, the algorithm iterates over the activities enabled in the current marking. If an activity is enabled and the enablement time of the next occurrence of this activity has not yet been initialized (the activity was not enabled before), then the enablement time of this activity is set to be equal to the current execution time. At this point, and given that the model can parse every trace in the (repaired) input log, the activity corresponding to event e must be enabled. Accordingly, the algorithm fires this activity and updates the current execution time to be equal to the end time of event e .

The algorithm relies on an auxiliary function *isEnabled*, which determines if a gateway is

enabled in the current marking of a BPMN model, as well as a function *fire*, which computes the marking reached from the current marking when firing a given gateway. These functions implement the semantics of gateways defined in the BPMN standard. Specifically, a split gateway (with a single incoming flow) is enabled when there is a token in its incoming flow. When it fires, the token is removed from its incoming flow and a token is produced in each of its outgoing flows (in case of an AND gateway) or in one of its outgoing sequence flows (in case of an XOR gateway). In the latter case, the conditional flow leading to enablement of the next event e in the trace is selected. Conversely, an AND-join gateway is enabled if there is a token in each of its incoming flows, while an XOR-join gateway is enabled when there is a token in any of its incoming flows. When a join gateway fires, the tokens in its incoming flows are removed and a token is added to its outgoing flow.

Discover the inter-arrival distribution. The goal here is to determine the Probability Distribution Function (PDF) of the inter-arrival times of cases. To this end, the traces in the log are sorted by the start time of the first activity in a trace.⁵ Then, the difference between the start time of a trace and the start time of the previous trace is calculated. The resulting data series of inter-case creation times is then analyzed in order to determine which PDF, among a set of available PDFs, yields the minimum standard error. The current implementation of the proposed method supports the following distribution functions: normal, exponential, uniform, fixed-value, triangular, gamma and log-normal. In the running example, we find that the PDF that best fits the observed inter-arrival times is an exponential PDF with a mean of 15455 seconds.

Conditional branching probabilities. The conditional branching probabilities can be directly calculated from the frequencies of traversal of the conditional branches computed during replay. For each decision gateway, the traversal frequencies of the outgoing conditional branches are normalized so that their sum is one, hence converting these traversal frequencies into (normalized) probabilities. In the case of our example event log, the XOR1 gateway has two possible paths to the activities “Amend Request for Quotation” and “Send Request for Quotation to Supplier”.

⁵ Here, we assume that the start timestamp of the first event in a trace corresponds to the time when the case was created. If this assumption does not hold, one can apply a pre-processing step to estimate the actual case creation time as discussed in [19]

These paths were executed in 563 and 608 occasions respectively which means execution probabilities of 0.48 and 0.52 of these paths as is shown in the Figure 4.

Figure 4: XOR gateway probabilities definition example

Activity processing times. The PDF of the processing time of a given activity A in the process model is determined in two steps. First, a data series consisting of observed processing time for each execution of activity A in the log (computed by the log replay) is created. Next, a collection of possible distribution functions to the data series is analyzed to select the distribution function that fits with the minimum standard error.

For example, each one of the 21 activities in the purchasing process event log were analyzed. As can be seen in the Table 3, most of the processing times of the activities follow a uniform distribution with a mean of 3600 seconds.

	PDF						
	Uniform	Normal	Exponential	Gamma	Lognorm	Fixed	Triangular
# of Activities	9	2	—	2	3	5	—
Mean	3600	1285.25	—	1027.55	764.66	24	—
StdDev	0	136.82	—	548.92	704.54	32.86	—

Table 3: Purchasing process activities probability distribution functions summary

Resource pools. Resource pools (which correspond to organizational roles or groups) are discovered by using the algorithm proposed in [20]. This algorithm is based on the definition of activity execution profiles for each resource. A clustering technique is then applied to identify groups of resources with similar activity execution profiles (i.e. groups of resources who generally perform the same types of activities). Each such cluster becomes a resource pool. The algorithm in [20] then assigns each activity to one or more resource pools. However, in a BPS model, each activity must be assigned to exactly one resource pool. Hence, we post-process the output of the algorithm in [20] to fulfil this property by assigning each activity to the pool that most frequently performs it.

In our running example, there are 26 resources. The algorithm in [20] identifies 5 resource

pools and assigned each activity to exactly one pool.⁶.

Simulation model assembly. Once we have compiled all the simulation parameters, we put them together with the BPMN model into a single data structure. This step is dependent on the target simulation tool (e.g. BIMP or Scylla). In BIMP for example, this step involves embedding the simulation parameters inside the BPMN model, using proprietary XML tags.

Simulate Process. In this last step, the BPS model is given as input to a process model simulator. The simulator outputs a simulated event log. Below we discuss how the accuracy of the resulting BPS model is assessed and optimized.

3.3. Assessment and optimization

The objective of this stage is to assess how accurate the event log generated in the simulation stage is in relation to the event log used as input. As mentioned before, it is possible to discover multiple versions of the process due to the characteristics of the process models discovery algorithms, which could be more or less precise in relation to the event log. To this end, we employ a Bayesian hyper-parameter optimizer to explore multiple hyper-parameter configurations with the goal of optimizing the accuracy of the resulting BPS model.

BPS model assessment. Assessing accuracy of a BPS model can be carried out subjectively or objectively. A subjective assessment depends on the interpretation done by experts about the outputs of the simulation (see Section 2). Therefore, the experts assess how reasonable the deviations of the simulation are in relation to the expected behavior, discarding the models with high deviation. However, the fact that unexpected behaviors are observed does not imply that they are not possible. This possibility makes the subjective evaluation risky and affected by a bias of the expert. An objective assessment depends of the quantitative comparison between the simulation values and the ground truth values. In this kind of validation is possible to perform statistical tests, such as the Student's t-test, which is used to rule out models that are not representative for the

⁶ It turns out that this event log contains information about roles and about the mapping from activities to roles. We found that the algorithm in [20] re-discovered the roles already present in the event log (without using this information) with 100% accuracy

explanation of the value of an objective variable. However, this test helps to reject models but not to select the best one.

Another possible quantitative evaluation is the distance measurement between the generated values and the ground truth. This kind of comparison is highly used in the area of predictive process monitoring, in which Demerau-Levinstein (DL) and Mean Absolute Error (MAE) have been used as distance metrics to assess the similarity between two process traces. DL is applied to measure the similarity of two discrete attribute sequences such as activity or resource name sequences. In practice to evaluate two sequences, these are string encoded allowing the measurement of how many insertion, deletion, substitution or transposition operations are necessary for the sequences to be the same. The fewer operations the sequences are performed closer. For its part, the MAE is a known metric that allows measuring the distance between a prediction and an expected value of a continuous variable. In the case of a trace the attributes to be evaluated are the execution times of the events or the cycle time of the trace. Although these metrics allow us to establish the precision of a model still do not provide a single measure to establish how accurate a simulation model is.

In addition to these metrics we adapted the model proposed in [21, 22] for measuring timed strings to the business process domain. In this article we present an extension for the strings distance metrics for sequences composed of symbols available solely in defined time intervals. In order to define our measure, we must first make some definitions.

Having A as the set of all possible process activities and R as the set of all possible process roles, L is the finite alphabet of all possible activity-role tuples in the process.

$$L = \{(a, r) \mid a \in A; r \in R\} \quad (1)$$

Similarly, we define an event as $e = (l, p, w)$, where l is a symbol taken from the alphabet L , p is the processing time and w the event waiting time, both belong to the set of real numbers $p, w \in \mathbb{R}$, and are non-negative. In this context $\mathcal{E}$ is the set of all events with their respective times.

$$\mathcal{E} = \{(l, p, w) \mid l \in L; p, w \in \mathbb{R}, p, w \geq 0\} \quad (2)$$

A trace is a non-empty sequence of events $\sigma = \langle e_1, e_2, \dots, e_n \rangle$, with $e_i = (l_i, p_i, w_i) \in \mathcal{E}, 1 \leq i \leq n$. The set of all process traces is $\mathcal{S}$. An event log $\mathcal{L}$ is a set of completed traces from $\mathcal{S}$ and $\mathcal{K}$ is the number of traces in the event log.

$$\mathcal{L} = \{\sigma_i \mid \sigma_i \in \mathcal{S}, 1 \leq i \leq \mathcal{K}\} \quad (3)$$

For the metric definition, we use the Demerau-Levinstein (DL) algorithm as a base. Lets suppose we have two traces $\sigma \in \mathcal{S}$ and $\sigma' \in \mathcal{S}'$ for which we denote $d(i, j)$ as the cost of the least cost trace from $\sigma\langle 0:i \rangle$ to $\sigma'\langle 0:j \rangle$ being $0 \leq i \leq |\sigma|$ and $0 \leq j \leq |\sigma'|$. Then we define the recursion function between two traces as:

$$d(i, j) = \min \begin{cases} 0 & \text{if } i = j = 0 \\ d(i-1, j) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle) & \text{if } i > 0 \\ d(i, j-1) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle) & \text{if } j > 0 \\ d(i-1, j-1) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle) & \text{if } i, j > 0 \\ d(i-2, j-2) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle) & \text{if } i, j > 1 \\ & \& \sigma\langle i \rangle = \sigma'\langle j-1 \rangle \\ & \& \sigma\langle i-1 \rangle = \sigma'\langle j \rangle \end{cases} \quad (4)$$

Each recursive call corresponds to the following cases:

- $d(i-1, j) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle)$ corresponds to an event deletion
- $d(i, j-1) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle)$ corresponds to an event insertion
- $d(i-1, j-1) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle)$ corresponds to an event match or mismatch
- $d(i-2, j-2) + c(\sigma\langle i \rangle, \sigma'\langle j \rangle)$ corresponds to an event transposition

As you can see the main variation to the DL algorithm is in addition to the function that calculates the cost of the operation that is typically 1 or 0. In this model assuming that there are two events $e = (l, p, w)$ and $e' = (l', p', w')$ the function that calculates the cost is defined as:

$$c(e, e') = \begin{cases} 1 & \text{if } l \neq l' \\ \beta_1 |p - p'| + (1 - \beta_1) |w - w'| & \text{if } l = l' \end{cases} \quad (5)$$

Where $|p - p'|$ is the absolute error of the normalized processing time and $|w - w'|$ is the absolute error of the normalized waiting time. On the other hand, the coefficient β_1 represents the significance of the processing time in the total duration of the event in the ground-truth. With this modification the DL algorithm allows to include a penalty related to the time difference in processing and waiting times providing a single measure of accuracy. In this paper we will call this measure Timed String Distance (TSD).

Hyper-parameter optimization. The accuracy of a simulation model depends to a large extent on the process structure that is used as the basis. However, when using discovery algorithms, it is possible to find multiple versions of the process from the same event log. This is because the algorithms seek a balance between interpretability and generalization. In the same way, this generates that the discovered models frequently do not represent all the behaviors registered in the event log, affecting the accuracy of the simulation model. In the case of the Split Miner algorithm, as explained in the Subsection 3.1, the η and ϵ parameters allow different structures of the process to be extracted by filtering the events relationships. However, determining what is the most accurate structure is a complex process of trial and error. Furthermore, the conformance between the event log and the model used as a base is also affected by the structure discovered, also affecting the accuracy of the model.

In a traditional approach, an expert would perform the search manually modifying the values of the parameters based on their expertise and intuition. However, this is a time-consuming approach that can lead to not finding the global optimum [23]. With this in mind, we propose to use a Tree-structured Parzen Estimator (TPE) as a hyper-parameter optimizer [24] to find the best settings based on historical accuracy the executed models. TPE is a sequential algorithm that, on each trial, defines the next parameter configuration based on historical results and nested functions that select the parameters values based on a probability distribution and ranges defined for each one. The objective function seeks to minimize the loss, calculated as the inverse TSD measure, and the search space considered is presented in Table 4

Category	Variable	Distribution	Range
Control flow discovery	Parallelism threshold (ϵ)	Uniform	[0...1]
	Percentile for frequency threshold (η)	Uniform	[0...1]
Log repair technique	Repair	n/a	
	Removal	n/a	
	Replace	n/a	

Table 4: Search space definition

4. Evaluation

The method described above has been implemented as an open-source tool, namely Simod, which is packaged as a Python Jupiter Notebook.⁷ Simod takes as input an event log in XES format, and produces as output a BPMN model that includes simulation parameters, ready to be simulated using the BIMP simulator [5].⁸ The source code of Simod can also be configured to produce models for the Scylla simulator [6], but BIMP is used as the default simulator because Scylla only supports a restricted set of probability distributions, thus restricting the space of configuration options.

Using Simod, we have conducted an experimental evaluation of the proposed BPS model discovery method, aimed at addressing the following research questions: (RQ1) What is the level of accuracy of the BPS models generated by the proposed method? and (RQ2) To what extent the hyper-parameter optimization step improves the accuracy of the resulting BPS simulation models?

4.1. Datasets

A pre-requisite to discover a BPS model from an event log, is that the events in the log should have both a start and end timestamps. Unfortunately, this pre-requisite is not fulfilled by publicly available real-life event logs such as those in the 4TU Collection of event logs.⁹ As an alternative, we validate the proposed approach using one synthetic event log and two real-life event logs: one coming from a Business Process Management System (BPMS) and one coming from a factory production system, which satisfy the above requirement. The descriptive statistics of these event logs can be found below, whereas their structure is described in Table 6.

The first event log is a synthetic log, generated from a model not available to the authors, of a purchase-to-pay (P2P) process. This is the same log used as a running example in Section 3.¹⁰ The second log stems from an Academic Credentials Recognition (ACR) process at University of Los Andes in Colombia. The log comes from a deployment of a Business Process Management System (BPMS), specifically Bizagi. The model corresponding to this log was not available to the

⁷ The tool and the event logs used in the evaluation are available at <https://github.com/AdaptiveBProcess/Simod>

⁸ Available at <http://bimp.cs.ut.ee>

⁹ https://data.4tu.nl/repository/collection:event_logs_real

¹⁰ The log is part of the academic material of the Fluxicon Disco tool – <https://fluxicon.com/>

authors of this article. The third log is that of a manufacturing production (MP) process, exported from an Enterprise Resource Planning (ERP) system [25]. The tasks in this process refer to steps (or “stations’;) in the manufacturing process.

Event log	Num. traces	Num. events	Num. activities	Avg. activities per trace	Max. activities per trace	Mean duration	Max. duration
P2P	608.0	9119.0	21.0	14.9	44.0	21.5 days	108 days 7 hours
ACR	954.0	6870.0	18.0	7.2	23.0	14.9 days	135 days 19 hours
MP	225.0	4953.0	26.0	22.0	177.0	20.6 days	87 days 10 hours

Table 5: Statistics of the event logs

These three event logs were chosen because they have distinct characteristics in terms of control-flow and they come from different domains. The ACR event log is the one that contains the greatest number of traces and the least number of average activities per trace, while MP is the log with the least traces and the highest number of average activities per trace. On the other hand, the P2P log corresponds to the scenario in which the event log is generated from a process model defined in ideal conditions, so it is expected a 100% fit between the two, and high accuracy in the simulation. The ACR log corresponds to a service process executed on a BPMS. It is a relatively complex process, which delivers a service to hundreds of users and involves over a dozen workers. Finally, the MP event log is the scenario in which the process structure is unknown, and where the behavior of the resources can affect the structure of the process significantly.

4.2. Experimental setup

For each event log we generate 100 BPS models with different setup combination of preprocessing parameters. The evaluated combinations were composed by variations of ϵ , η , and a log repairing method. The Simod hyper-parameter optimizer was used to choose the values of each combination. Each resulting simulation model was executed 5 times and new event logs with the same size of the original event log were generated. Subsequently, we evaluate the accuracy of the models as the distance between the simulated event log and the ground-truth event log using the TSD. To measure the distance of the entire event log, we paired each generated trace with the

most similar trace (w.r.t. TSD distance) of the ground-truth. Once the pairs (generated trace, ground-truth trace) were formed, we calculate the mean TSD between them. Additionally, we used a warm-up and cool-down of 0.2, and the results of each combination were averaged to find their convergence. In total, 100 combinations of hyper-parameters and 500 simulated event logs were evaluated for each input event log.

4.3. Results

Figure 5 presents the accuracy results of each configuration discriminated by event log. The figures in the first column show the relationship of the η (eta) values used in the discovery of the models in relation to their accuracy, the second column presents the ϵ (epsilon) values in relation to the accuracy, and the third column include both perspectives. Similarly, the plotted series represent the event log repair techniques (i.e. repair, replace or removal) applied in each configuration. Table 6 summarizes the most accurate BPS model generated for each event log (the one with the best TSD). The most accurate model among the three that are presented for each particular log repair method is highlighted. Additionally, the editing distance (DL) and Mean Absolute Error (MAE) are presented for each generated model as complementary measures.

Figure 5: Event logs accuracy evaluation

Event log	Log Repair Method	ϵ	η	TSD	DL	MAE (days)
P2P	repair	0.8701	0.2187	0.8214	0.9259	21.51
	replace	0.4569	0.4029	0.6073	0.7254	55.17
	remove	0.0655	0.1332	0.5952	0.6648	19.71
ACR	repair	0.1745	0.7648	0.9167	0.9376	10.33
	replace	0.6776	0.8782	0.8537	0.8845	12.21
	remove	0.2577	0.9609	0.8537	0.8918	13.30
MP	repair	0.6285	0.6664	0.3427	0.3907	18.91
	remove	0.8546	0.6728	0.3245	0.3590	11.41
	replace	0.6673	0.6726	0.3202	0.3858	7.23

MP expanded	repair	0.7839	0.6389	0.3558	0.4042	18.98
	remove	0.3031	0.6732	0.3484	0.3990	13.45
	replace	0.4034	0.7143	0.3189	0.3753	22.47

Table 6: Accuracy results summary

In the P2P event log, the repair technique always obtained the best accuracy results with a wide margin, and that is constant for the values of η greater than 0.14. Likewise, in the ACR event log it is observed how the values of η and ϵ have a clear influence on the models accuracy, especially those with η greater than 0.45 and ϵ lower than 0.5. Also, the repair technique is consistently better than the other two, although without a wide margin. In the case of the MP event log, two evaluation cycles were executed. In the first one, a complete search space evaluation was carried out, unveiling better accuracy results in η values greater than 0.4. More specifically, a peak in the models accuracy was observed in the η values of [0.60..0.72], and in the ϵ values of [0.30..1.00]. Therefore, in a second execution the search space was restricted to those intervals. The repair technique again obtained the highest accuracy but without a dominant trend, especially when compared with the removal technique.

The results when using different parameters for model discovering suggest that there is a positive correlation between the frequency threshold (η) and the accuracy of the discovered BPS model. For instance, the accuracy of the models discovered from the ACR event log improves as the values of η increase (cf. Figure 5). This η parameter plays a more significant role than the parallelism threshold (ϵ) for which there is no a clear trend.

The results of applying the log repair methods show an improvement in the accuracy of the BPS models. This effect can be observed in most of the cases, independently of the event log. Nevertheless, it is more evident in the P2P event log (synthetic log). The event logs with the lowest accuracy was MP, reaching a maximum of 0.35 in contrast to 0.82 and 0.91 of the other two event logs (see Table 6). This large difference may be related to the structure of the event log, since this is the one with the least number of traces and events available, as well as the largest number of activities per traces due to loops in the process. These characteristics mean that none of the techniques used can improve accuracy significantly, evidencing the need for more data to reduce the error.

5. Threat to validity

The experimental evaluation is restricted to one synthetic and two real-life event logs. As such, the generalizability of the results is limited: The results might be different for other event logs, and as shown in the evaluation, particularly for event logs for which the automated process discovery technique does not manage to discover an accurate process model.

Each parameter is extracted using a particular algorithm, because our focus was on automatic discovery of simulation models and the search for greater precision in relation to the process model used as the basis. One possible extension of this tool could include multiple extraction options for each parameter.

Due to the sequential nature of the BPS techniques, multitasking, batching and deliberate delays within a process due to relative priorities (e.g. a process being “low priority”), are not taken into account in the proposed approach. Addressing these problems would require the development of new simulation techniques that are beyond the scope of this work.

6. Conclusion and Future Work

This paper outlined a method for automated discovery of business process simulation models from event logs and defined a measure for assessing the accuracy of a BPS model relative to an event log. The proposed method takes as input an event log, automatically discovers a process model, aligns the log to the model (and repairs it accordingly), and applies a range of replay and organizational mining techniques to extract all the parameters required for simulation. Once a BPS model is discovered, its accuracy is measured using a timed string-edit distance between the simulation log(s) it generates and the original log. A hyper-parameter optimizer is then used to search through the space of possible configurations so as to maximize the accuracy of the final BPS model.

The proposed method has been implemented as an open-source tool, namely Simod, and evaluated using three real-life event logs from different domains. The evaluation shows that the hyper-parameter optimization method significantly improves the accuracy of the resulting BPS model, relative to an approach where default parameters are used. Also, it was observed that the best configuration found varies from one event log to another, further emphasizing the need for

automated hyper-parameter optimization in this setting.

The evaluation reported in this paper is limited in terms of number of datasets due to the difficulty in obtaining access to real-life event logs where every (human) activity has both a start and an end timestamp, which is essential in order to determine the processing times of the activities. A direction for future work is to conduct a more systematic evaluation using a larger set of event logs, so as to identify possible relations between the characteristics of an event log and the associated (optimal) hyper-parameter settings and thus derive guidance for the configuration of BPS models.

Another limitation of the present study is that it relies on a relatively simple approach to business process simulation, in which activity instances (a.k.a. work items) are assigned to resources on a first-in, first-out basis (no notion of prioritization), each resource only performs one work item at a time (no multi-tasking), and a resource starts a work item assigned to it immediately after the assignment and works on it uninterruptedly until it is completed (no delays due to fatigue effects, no pauses, and no batching). Extending the proposed approach to lift these limitations and studying the effects of these phenomena on the accuracy of BPS models is another avenue for future work,

Acknowledgments. This research is funded by the European Research Council (PIX Project).

References

- [1] M. Dumas, M. La Rosa, J. Mendling, H. A. Reijers, Fundamentals of Business Process Management, second edition Edition, Springer, 2018.
- [2] L. Marușter, N. R. T. P. van Beest, Redesigning business processes: a methodology based on simulation and process mining techniques, Knowl Inf Syst 21 (3) (2009) 267–297.
- [3] A. Rozinat, R. S. Mans, M. Song, W. M. P. van der Aalst, Discovering simulation models, Inform Syst 34 (3) (2009) 305–327.
- [4] N. Martin, B. Depaire, A. Caris, The Use of Process Mining in Business Process Simulation Model Construction, Bus Inf Syst Eng 58 (1) (2016) 73–87.
- [5] M. Abel, Lightning Fast Business Process Simulator, Master’s thesis, University of Tartu (2011).
- [6] L. Pufahl, T. Y. Wong, M. Weske, Design of an Extensible BPMN Process Simulator, in:

- E. Teniente, M. Weidlich (Eds.), *Proceedings of the Business Process Management Workshops*, Springer, 2017, pp. 782–795.
- [7] M. Camargo, M. Dumas, O. González-Rojas, Simod : A Tool for Automated Discovery of Business Process Simulation Models, in: B. Depaire, J. De Smedt, M. Dumas, D. Fahland, A. Kumar, H. Leopold, M. Reichert, S. Rinderle-Ma, S. Schulte, S. Seidel, W. van der Aalst (Eds.), *Proceedings of the Dissertation Award, Doctoral Consortium, and Demonstration Track at BPM 2019, CEUR Workshop Proceedings*, 2019, pp. 139–143.
 - [8] N. Martin, B. Depaire, A. Caris, The use of process mining in a business process simulation context: Overview and challenges, in: *IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, IEEE, 2014, pp. 381–388.
 - [9] M. T. Wynn, M. Dumas, C. J. Fidge, A. H. M. ter Hofstede, W. M. P. van der Aalst, Business Process Simulation for Operational Decision Support, in: A. ter Hofstede, B. Benatallah, H.-Y. Paik (Eds.), *Proceedings of the Business Process Management Workshops*, Springer, 2008, pp. 66–77.
 - [10] A. Rozinat, R. S. Mans, Mining CPN Models: Discovering Process Models with Data from Event Logs, in: *In Workshop and Tutorial on Practical Use of Coloured Petri Nets and the CPN*, 2006, pp. 57–76.
 - [11] I. Khodyrev, S. Popova, Discrete modeling and simulation of business processes using event logs, *Procedia Comput Sci* 29 (2014) 322–331.
 - [12] B. Gawin, B. Marcinkowski, How Close to Reality is the as-is Business Process Simulation Model?, *Organizacija* 48 (3) (2015) 155–176.
 - [13] A. Augusto, R. Conforti, M. Dumas, M. L. Rosa, Split miner: Discovering accurate and simple business process models from event logs, in: *2017 IEEE International Conference on Data Mining (ICDM)*, IEEE, 2017, pp. 1–10.
 - [14] A. Augusto, R. Conforti, M. Dumas, M. La Rosa, F. M. Maggi, A. Marrella, M. Mecella, A. Soo, Automated Discovery of Process Models from Event Logs: Review and Benchmark, *IEEE T Knowl Data En* 31 (4) (2018) 686–705.
 - [15] S. J. J. Leemans, D. Fahland, W. M. P. van der Aalst, Scalable process discovery and conformance checking, *Softw Syst Model* 17 (2) (2018) 599–631.
 - [16] A. Adriansyah, B. van Dongen, W. van der Aalst, Conformance checking using cost-based fitness analysis, in: *Proceedings of the International Conference on Enterprise Computing*

- (EDOC), IEEE, 2011, pp. 55–64.
- [17] A. Rogge-Solti, A. Senderovich, M. Weidlich, J. Mendling, A. Gal, In Log and Model We Trust? A Generalized Conformance Checking Framework, in: M. La Rosa, P. Loos, O. Pastor (Eds.), Proceedings of the International Conference on Business Process Management (BPM), Springer, 2016, pp. 179–196.
 - [18] R. M. Dijkman, M. Dumas, C. Ouyang, Semantics and analysis of business process models in BPMN, *Inform Software Tech* 50 (12) (2008) 1281–1294.
 - [19] N. Martin, B. Depaire, A. Caris, Using Event Logs to Model Interarrival Times in Business Process Simulation, in: M. Reichert, H. A. Reijers (Eds.), Proceedings of the Business Process Management Workshops, Springer, 2015, pp. 255–267.
 - [20] M. Song, W. M. P. van der Aalst, Towards comprehensive support for organizational mining, *Decis Support Syst* 46 (1) (2008) 300–317.
 - [21] S. Dobrišek, J. Žibert, N. Pavešić, F. Mihelič, An edit-distance model for the approximate matching of timed strings, *IEEE Trans. Pattern Anal. Mach. Intell.* 31 (4) (2009) 736–741.
 - [22] E. Asarin, N. Basset, A. Degorre, Distance on Timed Words and Applications, in: D. N. Jansen, P. Prabhakar (Eds.), Formal Modeling and Analysis of Timed Systems, Springer, 2018, pp. 199–214.
 - [23] N. Witt, C. Seifert, Understanding the Influence of Hyperparameters on Text Embeddings for Text Classification Tasks, in: J. Kamps, G. Tsakonas, Y. Manolopoulos, L. Iliadis, I. Karydis (Eds.), Research and Advanced Technology for Digital Libraries, Springer, 2017, pp. 193–204.
 - [24] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for Hyper-parameter Optimization, in: Proceedings of the 24th International Conference on Neural Information Processing Systems, Curran Associates Inc., 2011, pp. 2546–2554.
 - [25] D. Levy, Production analysis with process mining technology (2014). doi:10.4121/uuid:68726926-5ac5-4fab-b873-ee76ea412399.

Biography

Manuel Camargo is Junior Research Fellow of Information Systems at University of Tartu, Estonia, also is a Ph.D. student in Computer Sciences at University of Tartu, and Ph.D. student in Engineering at the Universidad de los Andes, Colombia. His research interests focus on business process simulation and deep learning techniques.

Marlon Dumas is a Professor of Information Systems at University of Tartu, Estonia. Prior to this appointment he was faculty member at Queensland University of Technology and visiting researcher at SAP Research, Australia. His research interests span across the fields of software engineering, information systems and business process management. He is co-author of the textbook “Fundamentals of Business Process Management”.

Oscar González-Rojas is Associate Professor at the Systems and Computing Engineering Department at the Universidad de los Andes in Colombia. He received a Ph.D. degree in Computer Science at the Vrije Universiteit Brussel (VUB), Belgium and at the Universidad de los Andes (Uniandes), Colombia in 2010. He is lecturer of Business Process Management (BPM) Governance and IT Governance courses in the Master in Business and Information Technology at Uniandes. His main actuation areas in research and consulting are BPM, IT Governance, and Enterprise Architecture.

Credit Author Statement

Manuel Camargo: Conceptualization, Methodology, Software, Writing- Original draft preparation, Investigation, Visualization, Validation **Marlon Dumas:** Supervision, Conceptualization, Writing- Reviewing and Editing, Validation. **Oscar González-Rojas:** Supervision, Conceptualization, Writing- Reviewing and Editing, Validation.

Highlights

- A method for automatic discovery of Business Process Simulation Models from event logs and its associated tool is proposed
- We propose a modified version of the DL distance, namely the Business Process Trace Distance for business process traces comparison and a similarity measure, namely Event Log Similarity (ELS) to compare two event logs.
- A hyper-parameter optimizer is used to find the best combination of discovery parameters to minimize the proposed ELS metric between a simulated event-log and the ground-truth.

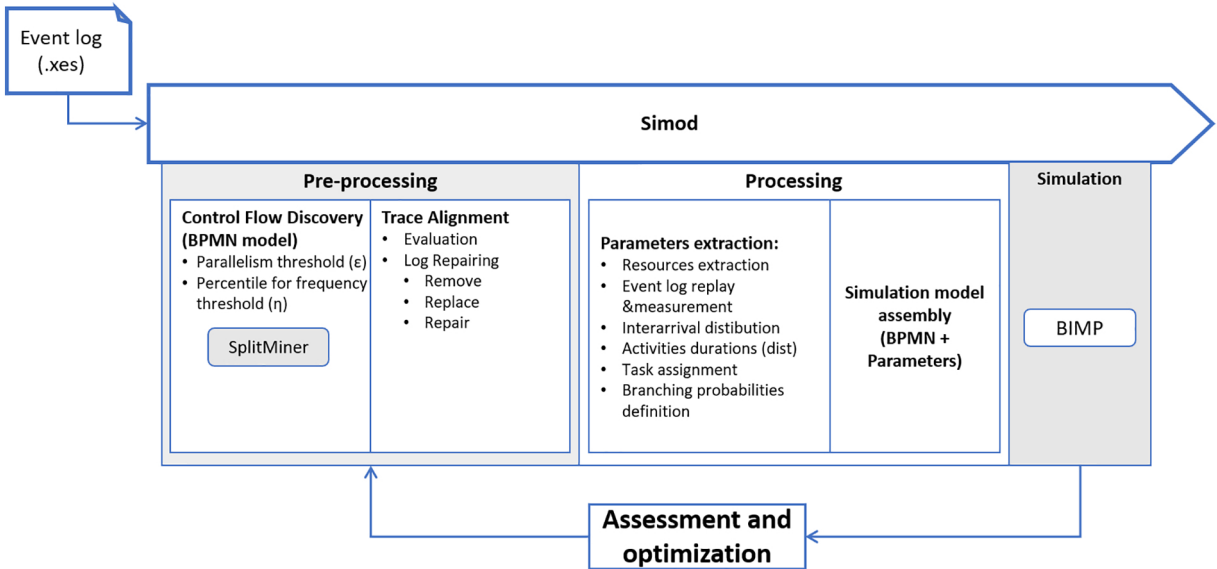


Figure 1

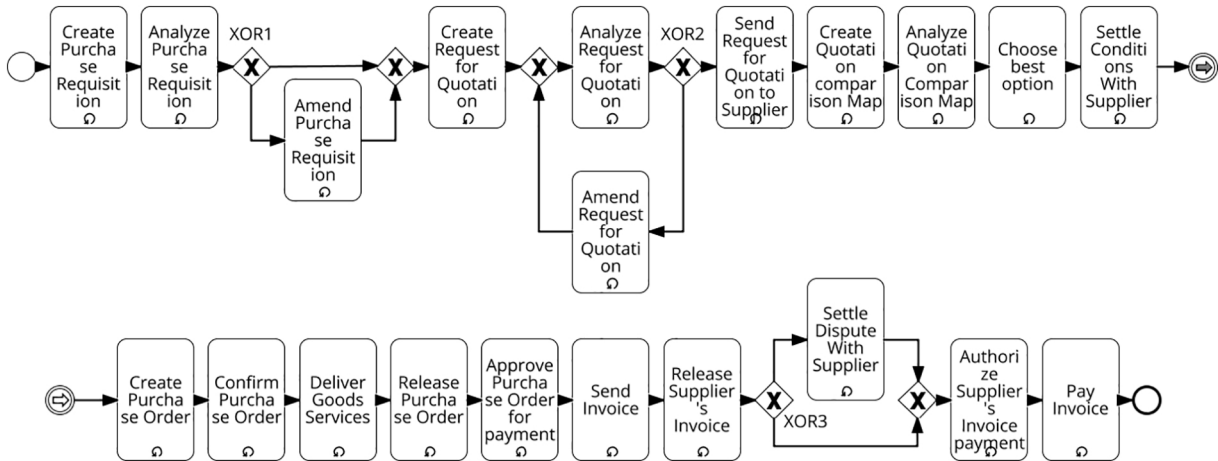


Figure 2

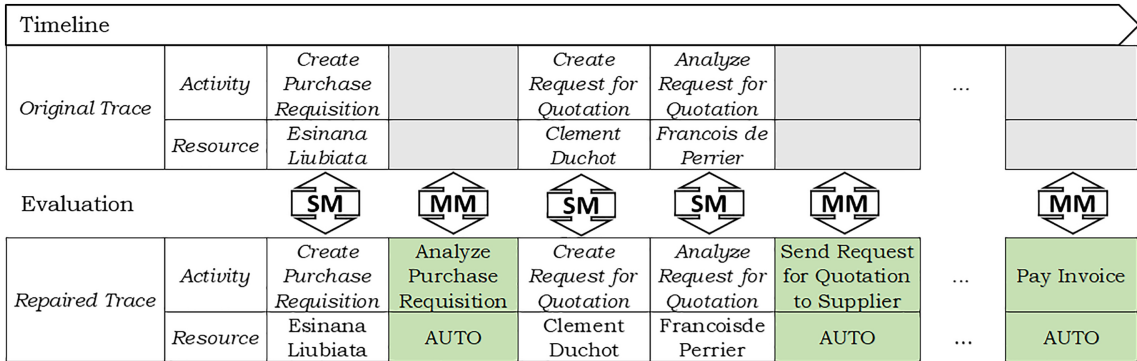


Figure 3

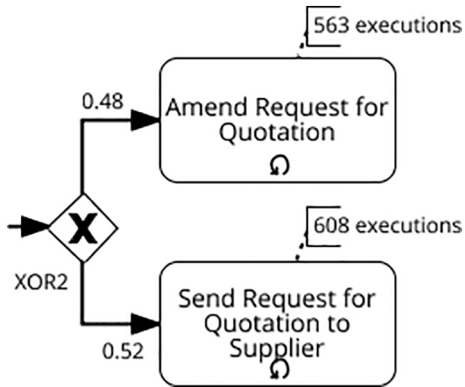


Figure 4

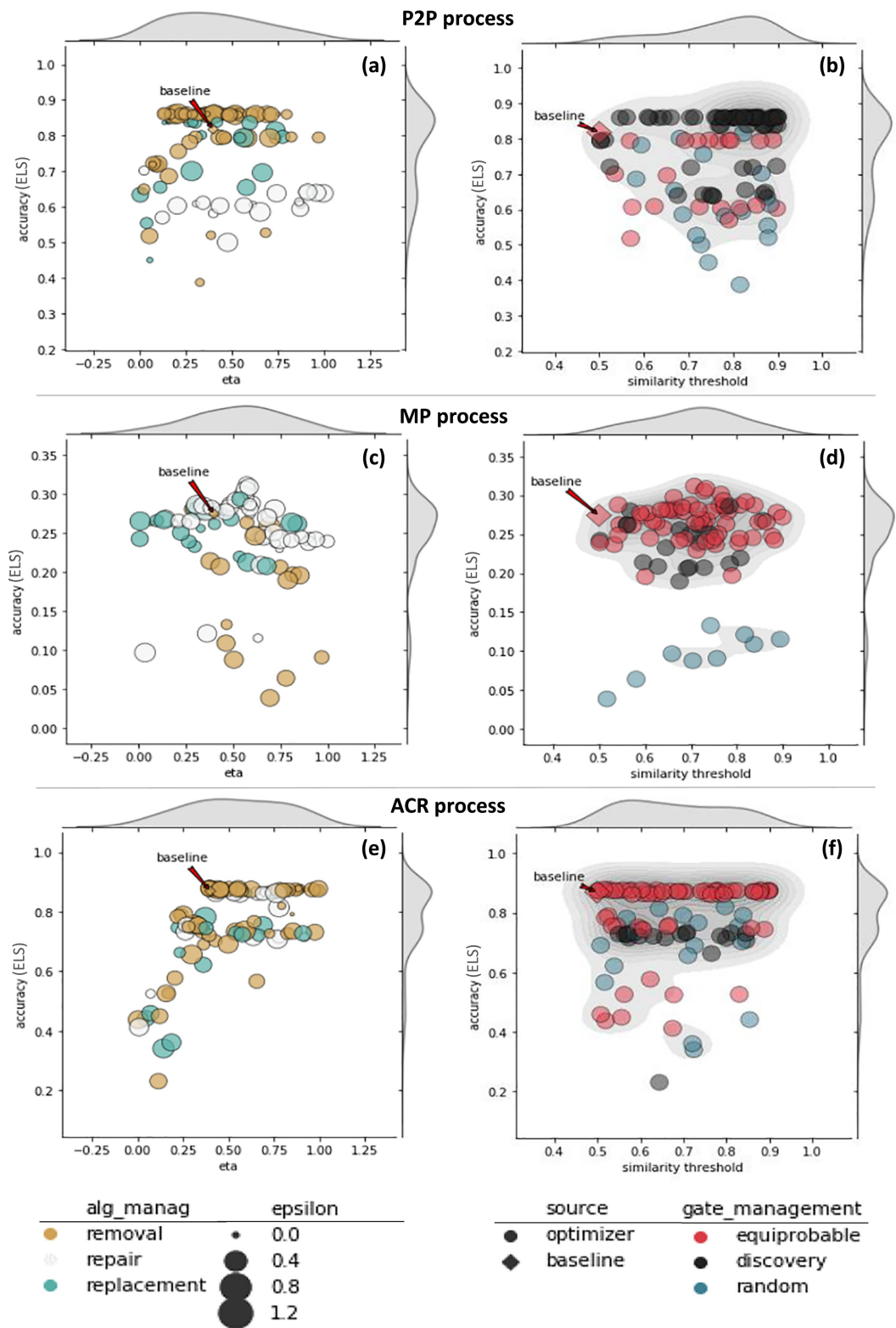


Figure 5